

Ford-Fulkerson Algorithm

Optimizing Network Augmented Paths and Residual Flow

1. Overview of Ford-Fulkerson

The **Ford-Fulkerson algorithm** is a method used to find the maximum possible flow in a flow network. It relies on the concept of **Augmenting Paths**—routes from the source to the sink that still have available capacity.

Method vs. Algorithm: Ford-Fulkerson is often called a "method" because it doesn't specify exactly *how* to find the augmenting path. If Depth-First Search (DFS) is used, it's standard Ford-Fulkerson; if Breadth-First Search (BFS) is used, it becomes the Edmonds-Karp algorithm.

2. The Residual Network & Back-Edges

To optimize flow, the algorithm maintains a **Residual Network**. This temporary graph tracks:

- **Forward Edges:** Remaining capacity (original capacity - current flow).
 - **Reverse Edges:** Backward capacity equal to the current forward flow.
- These allow the algorithm to "push back" flow and undo previous decisions to find a globally better path.

3. The Core Algorithm Steps

1. Initialize flow on all edges to 0.

2. While there is an augmenting path from source s to sink t in the residual network:
 - Find the **bottleneck capacity** (minimum residual capacity along the path).
 - For each edge in the path, increase flow by the bottleneck value.
 - Update the residual capacities (decrease forward, increase backward).
3. Return the total flow accumulated.

4. Python Implementation Snippet

```
def ford_fulkerson(self, source, sink):
    max_flow = 0
    while True:
        path = self.dfs(source, sink) # Find path with capacity > 0
        if not path: break

        # 1. Bottleneck
        bottleneck = min(self.adj[u][v] for u, v in zip(path, path[1:]))

        # 2. Update Residuals
        for i in range(len(path) - 1):
            u, v = path[i], path[i+1]
            self.adj[u][v] -= bottleneck # Forward
            self.adj[v][u] += bottleneck # Backward (fictitious edge)

        max_flow += bottleneck
    return max_flow
```

5. Time Complexity: $O(E * f)$

The complexity of Ford-Fulkerson depends on the maximum flow f . In the worst case, if each path only increases flow by 1 unit, the algorithm will run f times.

Complexity: $O(E * f)$, where E is the number of edges. This can be problematic if edge capacities are very large integers.

CodeHive

Advanced Depth-First Search Selection (Part 1)

Using DFS to find augmenting paths is the standard implementation of Ford-Fulkerson. However, DFS does not guarantee the *shortest* path in terms of edge count. This can lead to a phenomenon where the algorithm "zig-zags" through the graph, consuming many iterations before reaching the sink.

Path Reversal Logic: The introduction of reversed edges with flow values is what allows Ford-Fulkerson to be "self-correcting." Without these edges, a greedy choice that fills a central edge might block two better paths, but the back-edge allows the algorithm to retract that flow.

Termination and Integrity

The algorithm terminates when no path exists from S to T in the residual graph. If the graph capacities are irrational, Ford-Fulkerson might never terminate. For most practical applications (integer capacities), it always converges to the maximum flow, fulfilling the Max-Flow Min-Cut theorem.

Advanced Depth-First Search Selection (Part 2)

Using DFS to find augmenting paths is the standard implementation of Ford-Fulkerson. However, DFS does not guarantee the *shortest* path in terms of edge count. This can lead to a phenomenon where the algorithm "zig-zags" through the graph, consuming many iterations before reaching the sink.

Path Reversal Logic: The introduction of reversed edges with flow values is what allows Ford-Fulkerson to be "self-correcting." Without these edges, a greedy choice that fills a central edge might block two better paths, but the back-edge allows the algorithm to retract that flow.

Termination and Integrity

The algorithm terminates when no path exists from S to T in the residual graph. If the graph capacities are irrational, Ford-Fulkerson might never terminate. For most practical applications (integer capacities), it always converges to the maximum flow, fulfilling the Max-Flow Min-Cut theorem.

Advanced Depth-First Search Selection (Part 3)

Using DFS to find augmenting paths is the standard implementation of Ford-Fulkerson. However, DFS does not guarantee the *shortest* path in terms of edge count. This can lead to a phenomenon where the algorithm "zig-zags" through the graph, consuming many iterations before reaching the sink.

Path Reversal Logic: The introduction of reversed edges with flow values is what allows Ford-Fulkerson to be "self-correcting." Without these edges, a greedy choice that fills a central edge might block two better paths, but the back-edge allows the algorithm to retract that flow.

Termination and Integrity

The algorithm terminates when no path exists from S to T in the residual graph. If the graph capacities are irrational, Ford-Fulkerson might never terminate. For most practical applications (integer capacities), it always converges to the maximum flow, fulfilling the Max-Flow Min-Cut theorem.